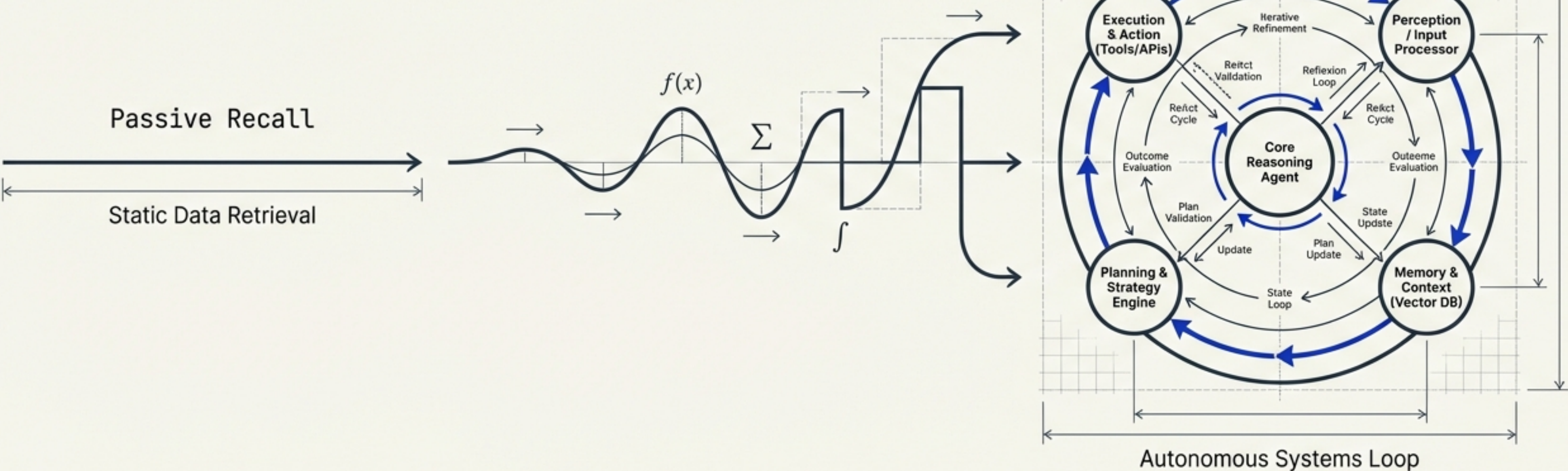


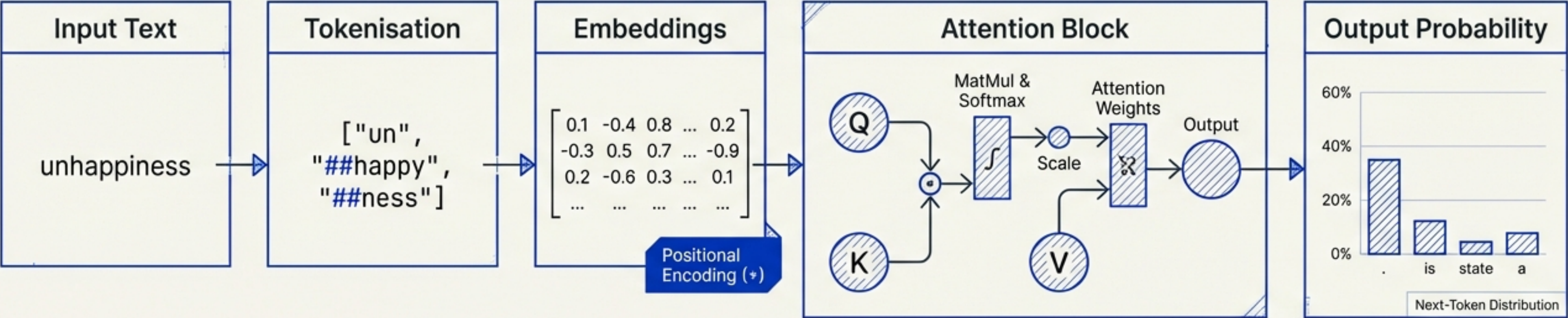
Agentic AI: From Passive Recall to Active Reasoning

A comprehensive blueprint for architects moving beyond chatbots to autonomous systems.

This deck explores the transition from Passive AI (Oracles that answer questions) to Agentic AI (Employees that reason, plan, and act). We will deconstruct the cognitive architecture, reasoning loops (ReAct/Reflexion), and the modern engineering stack required to build them.



The Engine: Embeddings, Attention, and Prediction



Tokenisation

Breaks text into sub-words to manage vocabulary efficiency.

"unhappiness" → ['un', '##happy', '##ness'].

Embeddings

Converts tokens into high-dimensional vectors (768–4096 dims).
Mathematically clusters concepts like "cat" and "dog".

Self-Attention

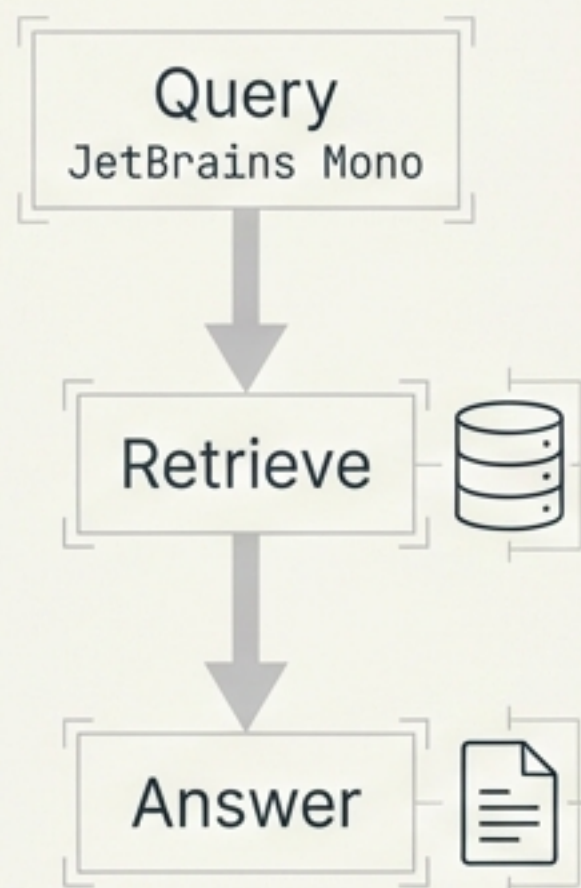
The mechanism where words "look at" each other.
Resolves that "it" refers to "animal" (45% relevance) in the sentence "The animal didn't cross the street because it was tired".

Positional Encoding

Mathematical tags (Sine/Cosine or RoPE) added so the model distinguishes "The cat sat" from "Sat cat the".

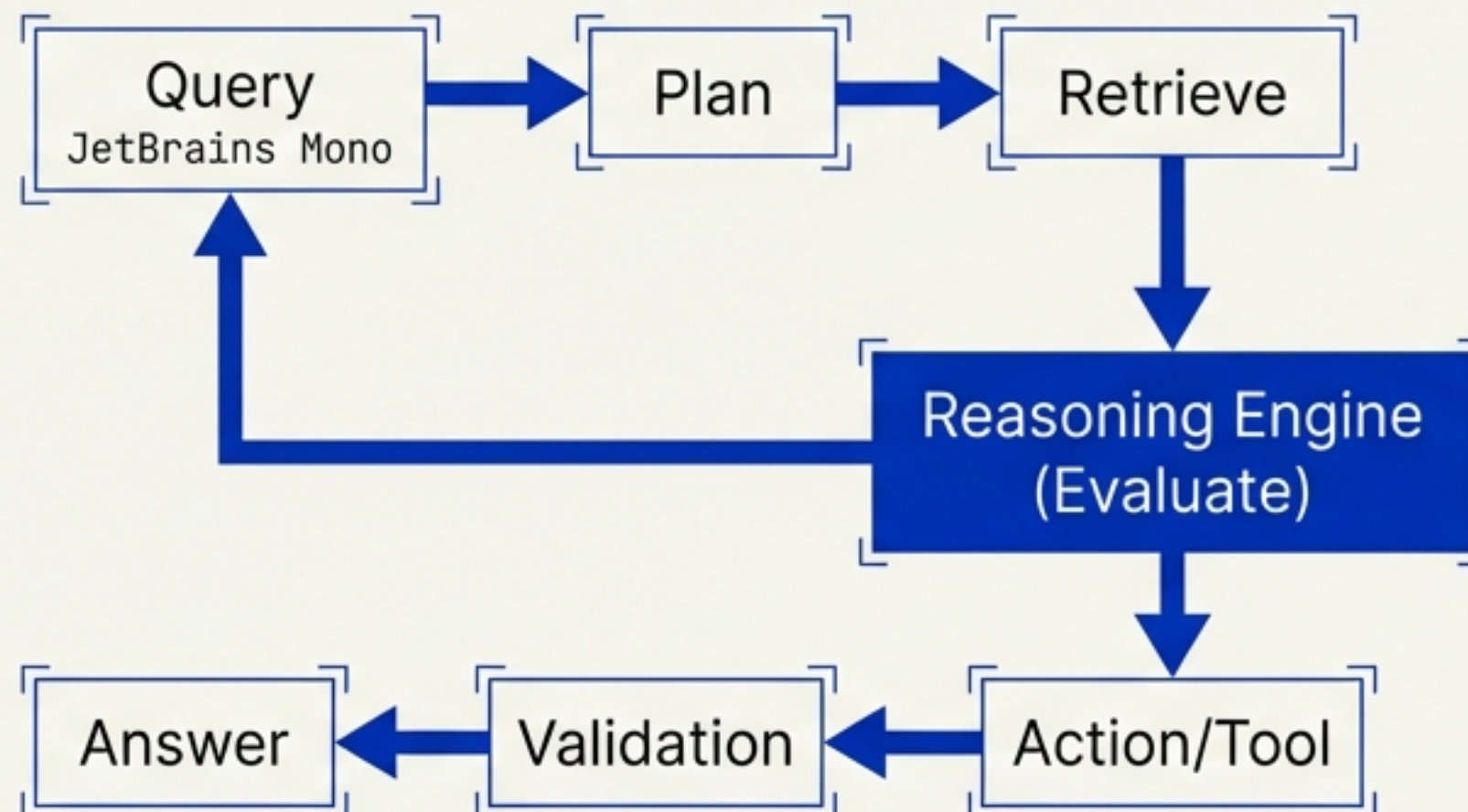
The Architectural Shift: Linear vs. Cyclic

Standard RAG



Retrieves once. No follow-up. Struggles with multi-step reasoning. If retrieval fails, the answer is hallucinated or incomplete.

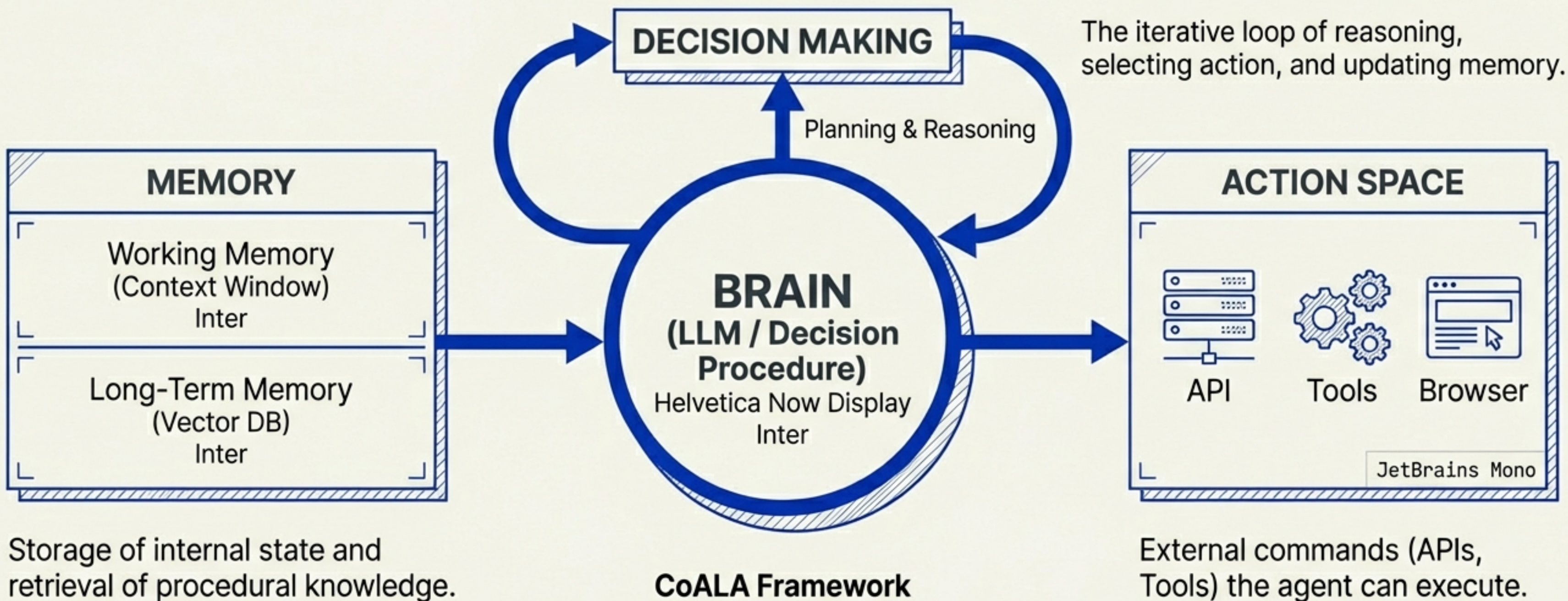
Agentic RAG



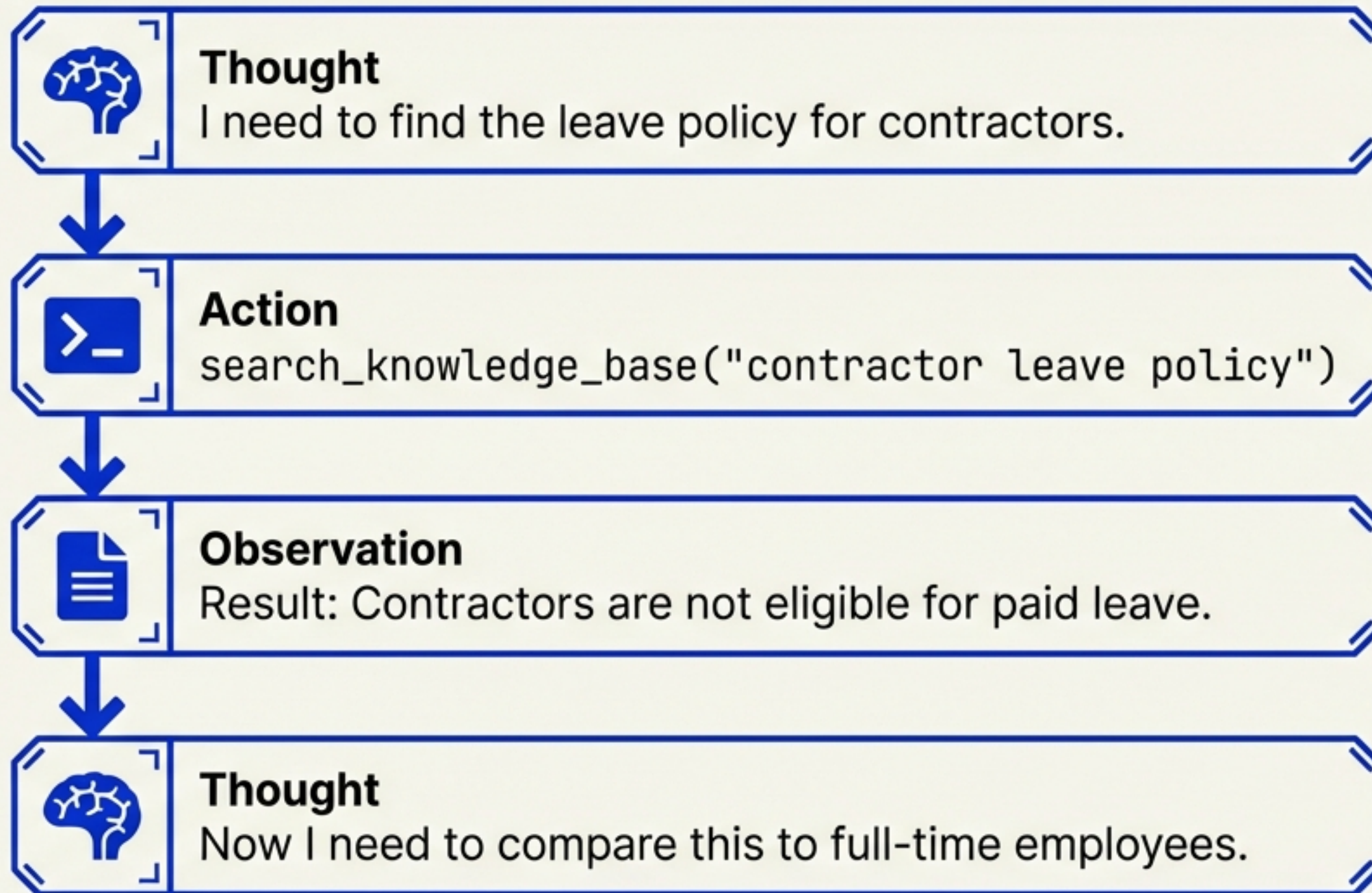
Uses a Reasoning Engine to evaluate data sufficiency. Creates a plan, executes actions, and loops until the goal is met. Adapts to uncertainty dynamically.

The Anatomy of an Agent (CoALA Framework)

An agent is a system composed of modular memory, a structured action space, and a generalised decision-making process.



The Reasoning Engine: ReAct (Reason + Act)

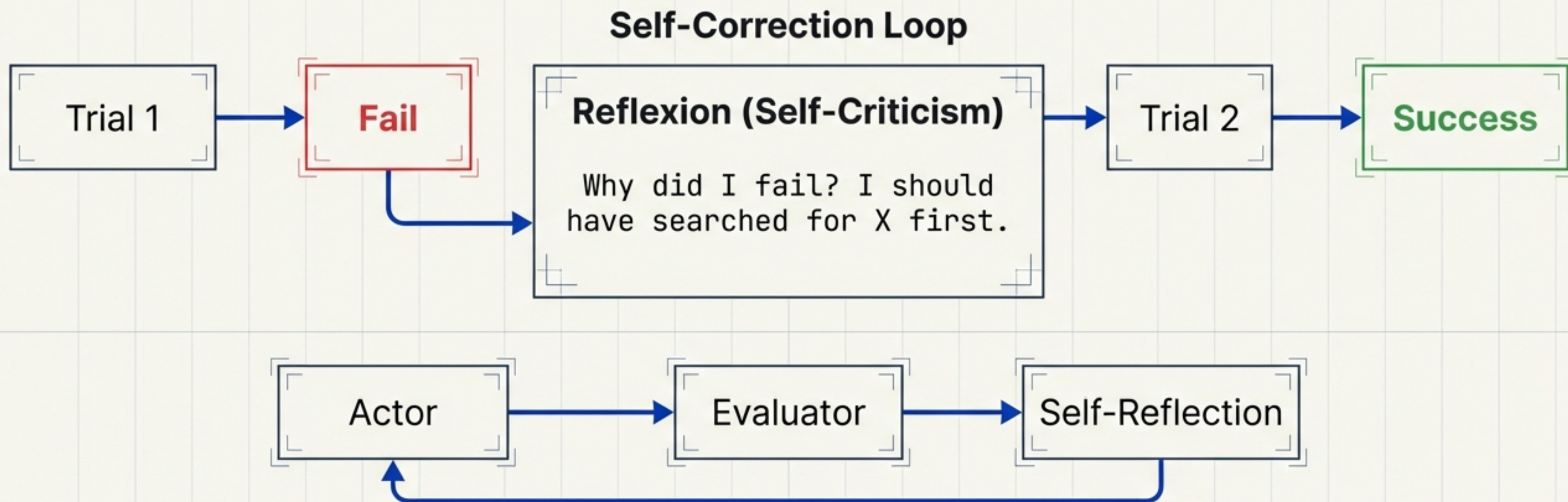


ReAct synergises reasoning (Chain of Thought) with acting (Tool use). It grounds reasoning in external reality through observations, reducing hallucinations.

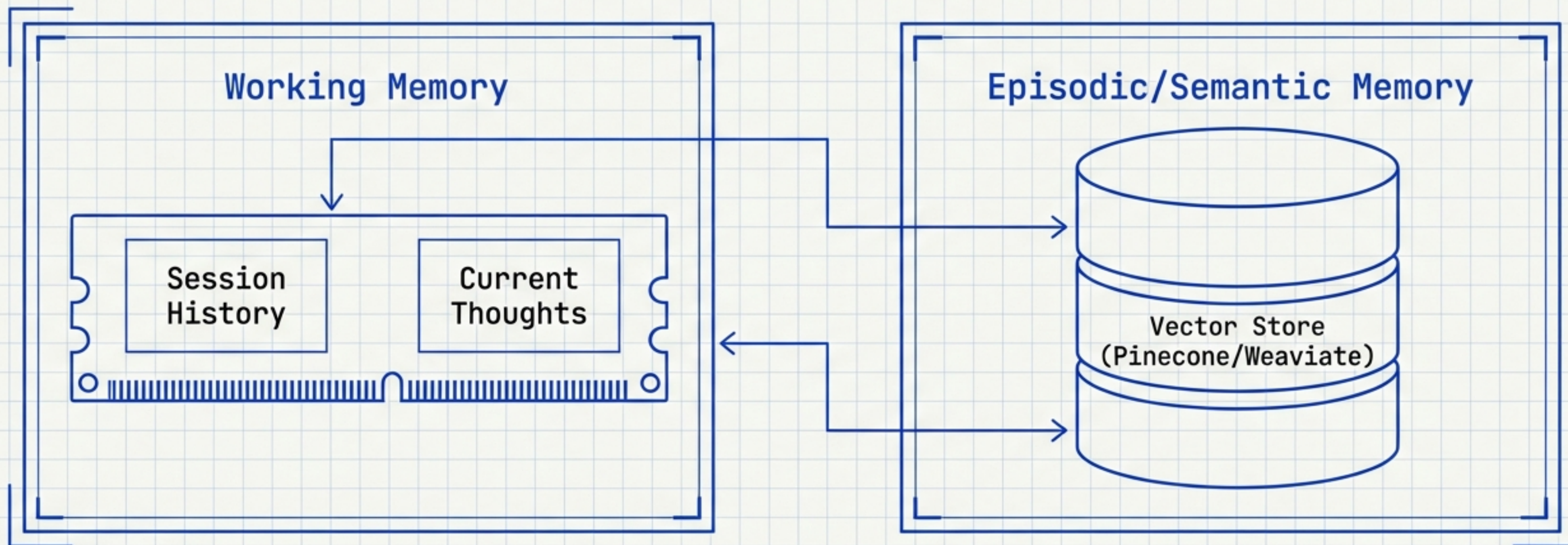
Self-Correction: The Reflexion Pattern

Instead of updating model weights (training), Reflexion updates the context (memory). The agent stores a "lesson" to guide the next attempt.

AlfWorld Benchmark: ReAct + Reflexion completed 130/134 tasks vs. ReAct alone.



The Memory Module: Context and Continuity



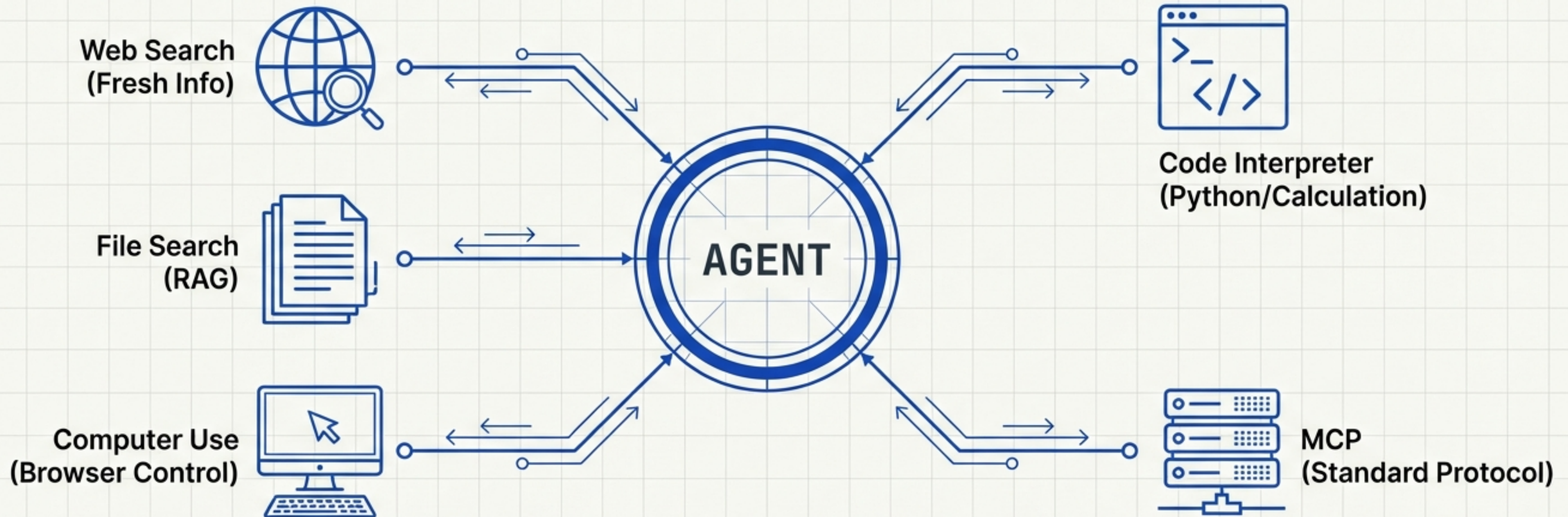
Short-Term: In-context learning. Limited by context window.

Long-Term: Retrieval of past experiences or procedural knowledge via RAG.

```
agent.memory.save_context("user_name", "Zylker")
```

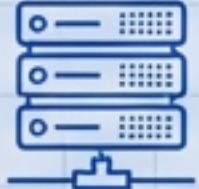
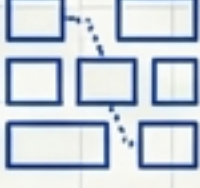
The Tool Layer: Hands and Eyes

Insight: Reasoning is limited without the ability to affect the world. Tools bridge the gap between passive knowledge and active execution.



The Modern Stack: Response API vs. Chat Completions

Insight: Moving away from stateless 'text-in/text-out' towards managed, event-driven agent orchestration.

	Chat Completions (Legacy)	Response API (Agentic)
State Management	 Stateless (Resend full history)	 Native State (Managed by platform)
Tooling	 Manual (Parse & Loop)	 Built-in (File Search, Code Exec)
Streaming	 Raw Text Chunks	 Semantic Events (ResponseCreatedEvent)

Building with the OpenAI Agentic SDK

```
from openai import AssistantAgent, tool
```

```
# 1. Define Tools
```

```
@tool
```

Decorator registers function for LLM use.

```
def get_weather(city: str) -> str:
```

```
    """Returns current weather for a city."""
```

```
    return f"The weather in {city} is 29°C and sunny."
```

```
# 2. Initialise Agent
```

```
agent = AssistantAgent(
```

```
    name="WeatherBot",
```

```
    tools=[get_weather],
```

```
    model="gpt-4o"
```

```
)
```

Defines model and tool access.

```
# 3. The Control Loop (Abstracted)
```

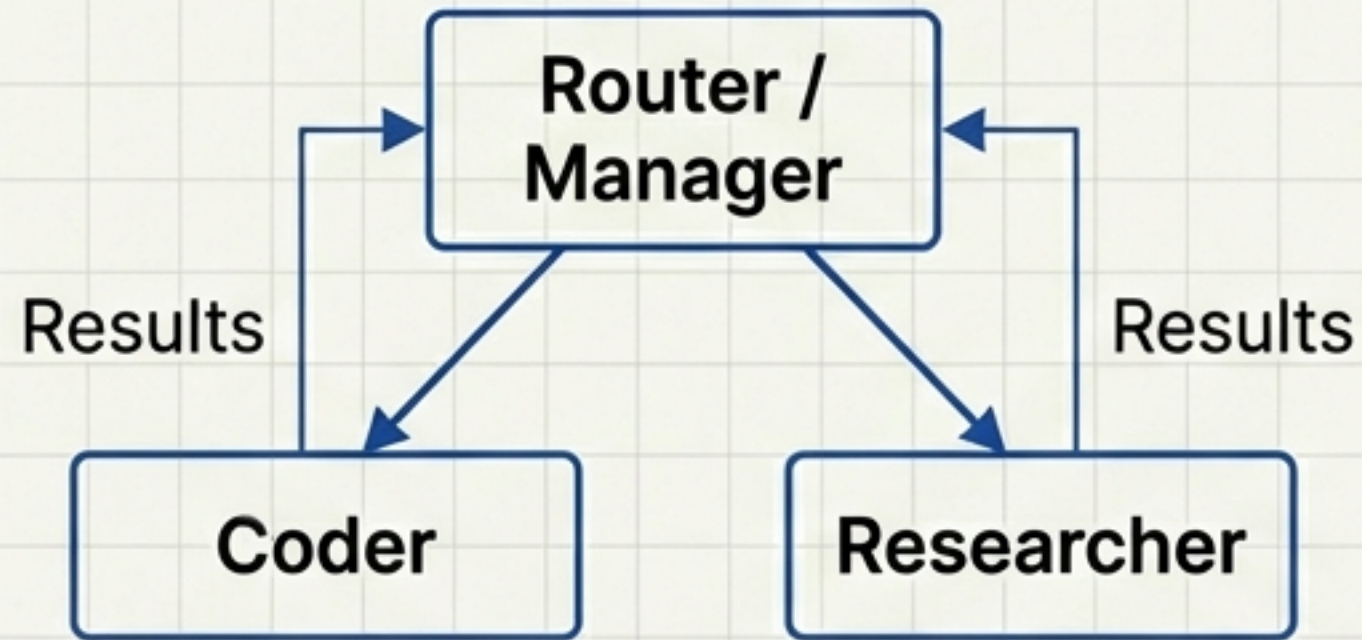
```
response = agent.chat("What's the weather in Miami?")
```

```
print(response)
```

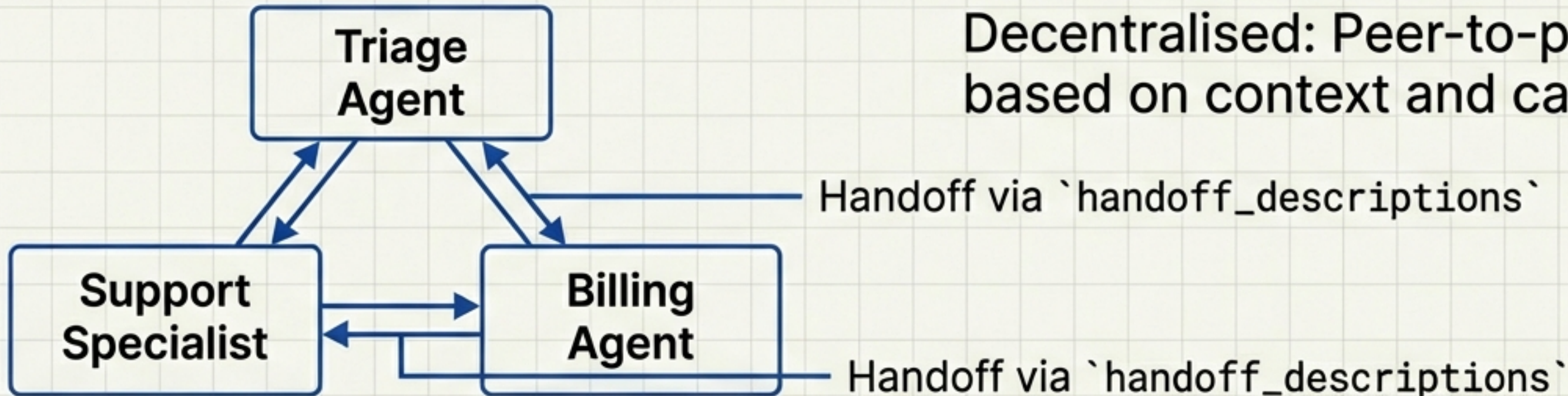
SDK handles the decision/execution loop automatically.

Multi-Agent Patterns: Collaboration

Collaboration and Handoffs



Hierarchical: Manager delegates tasks to specialists.



Decentralised: Peer-to-peer routing based on context and capabilities.

Engineering Guardrails and Safety

Autonomy requires boundaries. Protecting against infinite loops, API abuse, and prompt injection.



Rate Limiting

Throttles API requests to prevent abuse.



Input/Output Validation

Sanitizes data to prevent prompt injection.



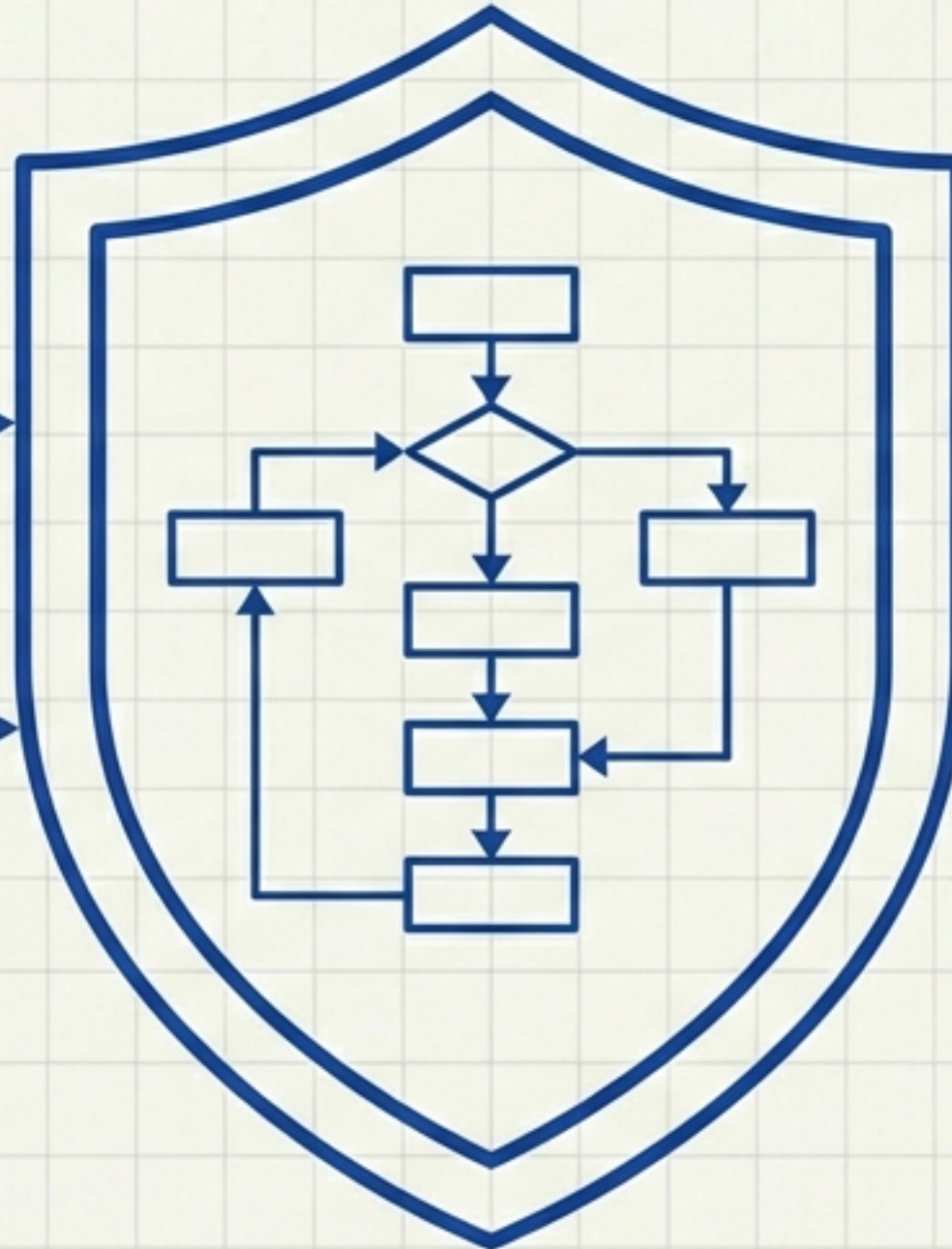
Human-in-the-Loop

Requires manual approval for critical actions.



Loop Prevention

Detects and stops infinite execution cycles.



Real-World Applications

Customer Support



Dynamic Policy Lookup.

Comparing multiple documents (Contractor vs. Full-time) to synthesise answers.

Data Analysis



AI Copilots.

Autonomously writing SQL queries, fetching data, and generating Python visualisations.

Workflow Automation



Task Execution.

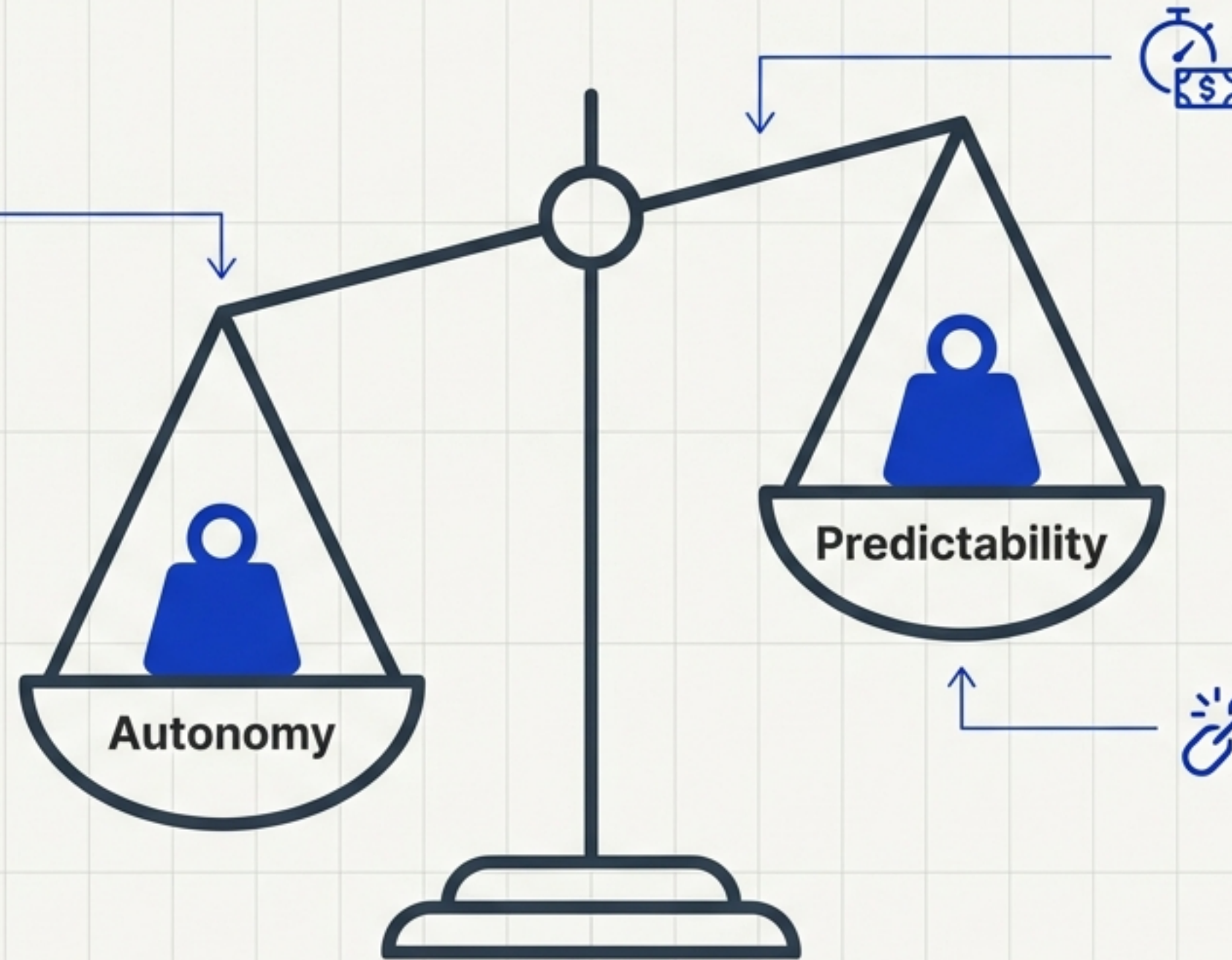
Reading emails, planning tasks, and executing CRM API calls without intervention.

Challenges and Evaluation

Evaluation



- Traditional metrics (BLEU/ROUGE) fail.
- Need functional benchmarks like AgentBench & ALFWorld.



Cost & Latency



- Multiple model calls per query increase expense.
- The "\$20,000/month agent" concept.

Reliability



- Probabilistic nature leads to occasional plan abandonment or loops.

